



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Iana Kim Afilleje
Section:	AN42
Professor:	Ms Crisola
Date:	9/28/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

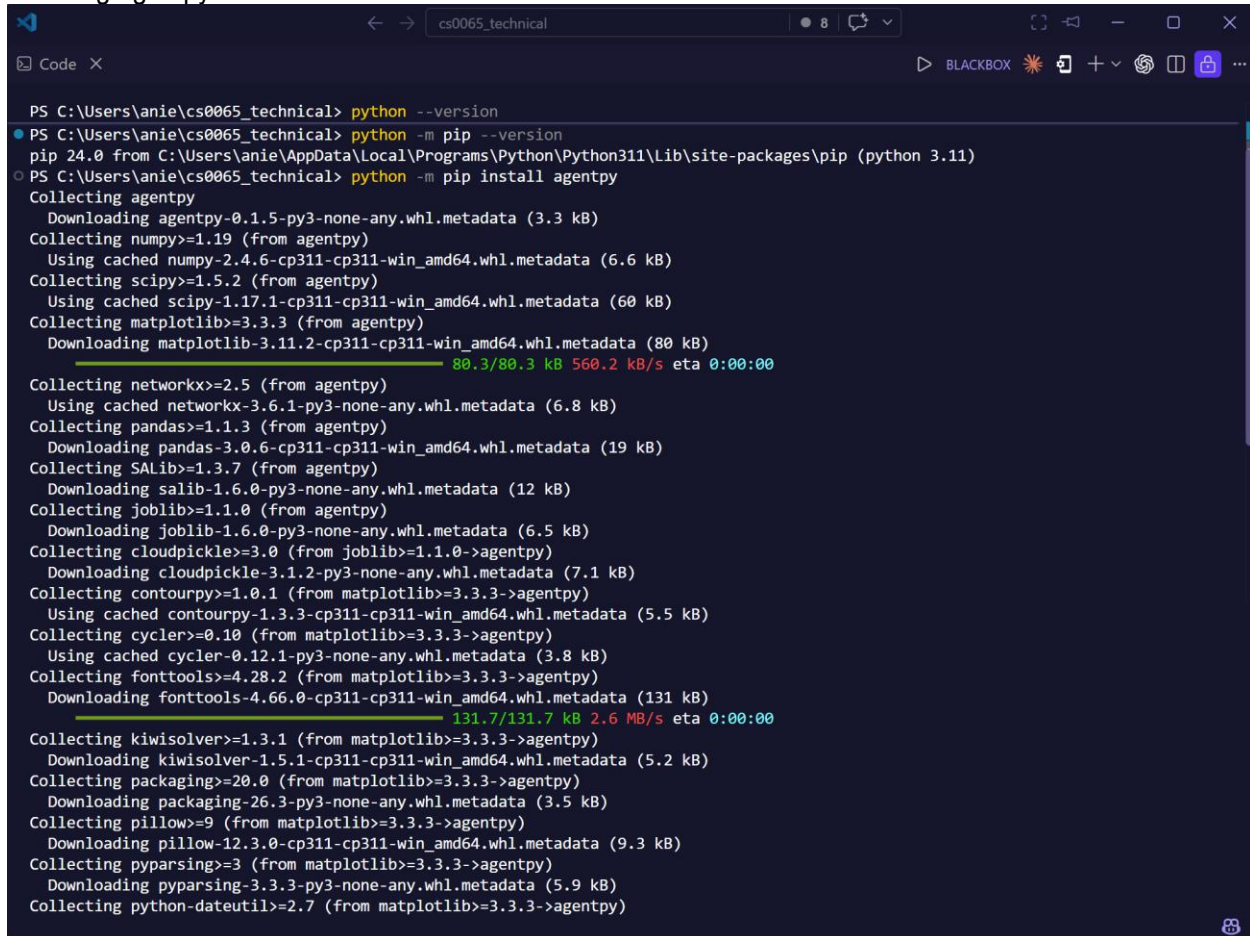
General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
(Example: TA1_Patilano.pdf)

8. Submit the **python** file and **PDF** file.

Screenshots:

Installing agentpy



```
PS C:\Users\anie\cs0065_technical> python --version
Python 3.11.0
PS C:\Users\anie\cs0065_technical> python -m pip --version
pip 24.0 from C:\Users\anie\AppData\Local\Programs\Python\Python311\Lib\site-packages\pip (python 3.11)
PS C:\Users\anie\cs0065_technical> python -m pip install agentpy
Collecting agentpy
  Downloading agentpy-0.1.5-py3-none-any.whl.metadata (3.3 kB)
Collecting numpy>=1.19 (from agentpy)
  Using cached numpy-2.4.6-cp311-cp311-win_amd64.whl.metadata (6.6 kB)
Collecting scipy>=1.5.2 (from agentpy)
  Using cached scipy-1.17.1-cp311-cp311-win_amd64.whl.metadata (60 kB)
Collecting matplotlib>=3.3.3 (from agentpy)
  Downloading matplotlib-3.11.2-cp311-cp311-win_amd64.whl.metadata (80 kB)
    80.3/80.3 kB 560.2 kB/s eta 0:00:00
Collecting networkx>=2.5 (from agentpy)
  Using cached networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting pandas>=1.1.3 (from agentpy)
  Downloading pandas-3.0.6-cp311-cp311-win_amd64.whl.metadata (19 kB)
Collecting SALib>=1.3.7 (from agentpy)
  Downloading salib-1.6.0-py3-none-any.whl.metadata (12 kB)
Collecting joblib>=1.1.0 (from agentpy)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.1.0->agentpy)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Collecting contourpy>=1.0.1 (from matplotlib>=3.3.3->agentpy)
  Using cached contourpy-1.3.3-cp311-cp311-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib>=3.3.3->agentpy)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib>=3.3.3->agentpy)
  Downloading fonttools-4.66.0-cp311-cp311-win_amd64.whl.metadata (131 kB)
    131.7/131.7 kB 2.6 MB/s eta 0:00:00
Collecting kiwisolver>=1.3.1 (from matplotlib>=3.3.3->agentpy)
  Downloading kiwisolver-1.5.1-cp311-cp311-win_amd64.whl.metadata (5.2 kB)
Collecting packaging>=20.0 (from matplotlib>=3.3.3->agentpy)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib>=3.3.3->agentpy)
  Downloading pillow-12.3.0-cp311-cp311-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib>=3.3.3->agentpy)
  Downloading pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Collecting python-dateutil>=2.7 (from matplotlib>=3.3.3->agentpy)
```

```
PS C:\Users\anie\cs0065_technical> python -m pip install agentpy
Using cached contourpy-1.3.3-cp311-win_amd64.whl (225 kB)
Using cached cycycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.66.0-cp311-cp311-win_amd64.whl (1.6 MB)
 1.6/1.6 MB 2.4 MB/s eta 0:00:00
Downloading kiwisolver-1.5.1-cp311-cp311-win_amd64.whl (70 kB)
 70.7/70.7 kB 2.0 MB/s eta 0:00:00
Downloading packaging-26.3-py3-none-any.whl (129 kB)
 130.0/130.0 kB 1.9 MB/s eta 0:00:00
Downloading pillow-12.3.0-cp311-cp311-win_amd64.whl (7.2 MB)
 7.2/7.2 MB 2.0 MB/s eta 0:00:00
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
 126.4/126.4 kB 1.8 MB/s eta 0:00:00
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading multiprocessing-0.70.19-py311-none-any.whl (144 kB)
 144.5/144.5 kB 2.9 MB/s eta 0:00:00
Downloading tzdata-2026.4-py2.py3-none-any.whl (347 kB)
 347.5/347.5 kB 1.4 MB/s eta 0:00:00
Downloading dill-0.4.1-py3-none-any.whl (120 kB)
 120.0/120.0 kB 1.7 MB/s eta 0:00:00
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: tzdata, six, pyparsing, pillow, packaging, numpy, networkx, kiwisolver, fonttools, dill, cycycler, clou
dpickle, scipy, python-dateutil, multiprocessing, joblib, contourpy, pandas, matplotlib, SALib, agentpy
Successfully installed SALib-1.6.0 agentpy-0.1.5 cloudpickle-3.1.2 contourpy-1.3.3 cycycler-0.12.1 dill-0.4.1 fonttools-4.66.0 joblib-
1.6.0 kiwisolver-1.5.1 matplotlib-3.11.2 multiprocessing-0.70.19 networkx-3.6.1 numpy-2.4.6 packaging-26.3 pandas-3.0.6 pillow-12.3.0 p
yparsing-3.3.3 python-dateutil-2.9.0.post0 scipy-1.17.1 six-1.17.0 tzdata-2026.4

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\anie\cs0065_technical> python -c "import agentpy; print('AgentPy installed')"
AgentPy installed
PS C:\Users\anie\cs0065_technical>
```

Testing the code:

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
```

```

        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

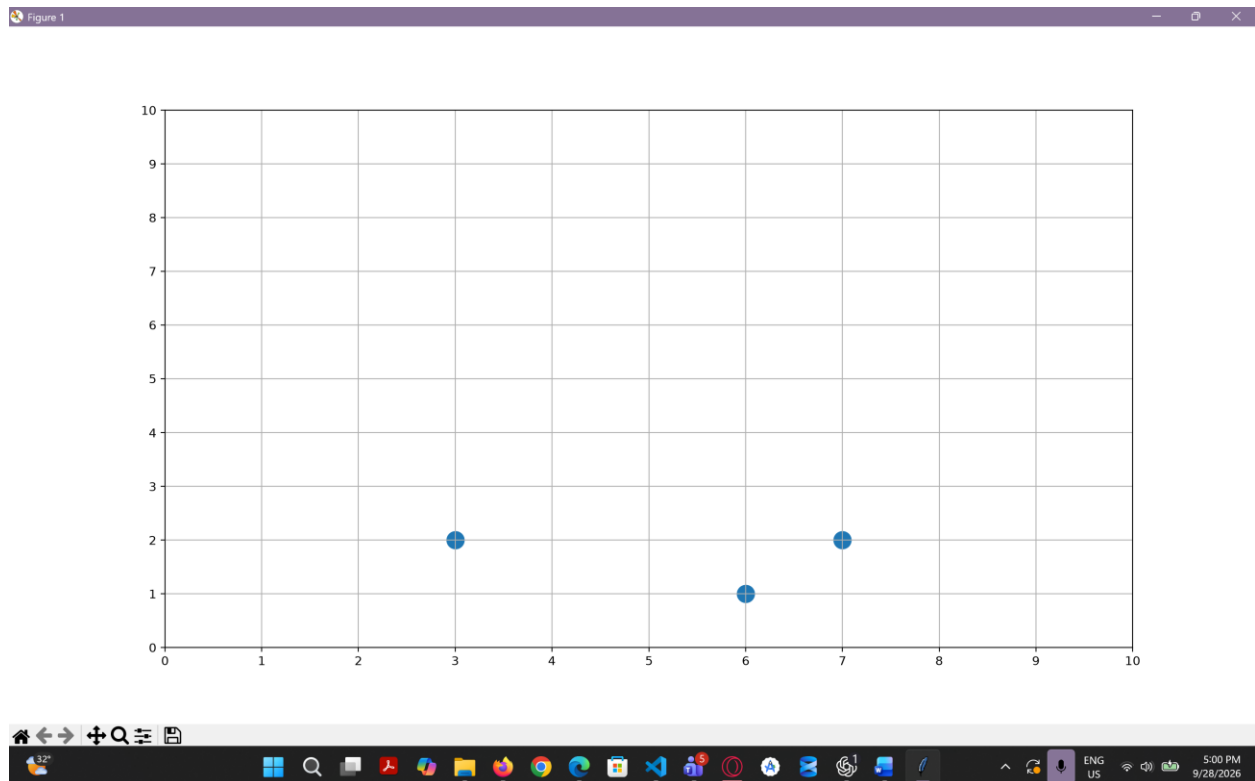
def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.

```
PS C:\Users\anie\cs0065_technical> python -u .\sample.py
Enter number of agents:
3
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 15
█
```



3. Enter the following values when prompted:

- **Number of agents** (e.g., 5)
- **Grid size** (e.g., 10 for a 10x10 grid)
- **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↓↓

```
import agentpy as ap
import random
import numpy as np
import matplotlib.pyplot as plt

# =====
# Task 1 & 2: Agent Definition with Path Tracking & Avoidance
# =====
class AdvancedWalker(ap.Agent):
    def setup(self):
        # Task 1: Initialize list to track path history
        self.path = []

    def step(self):
        # Record current position before attempting a move
        self.path.append(self.position)

        # Directions: Up, Down, Left, Right
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        random.shuffle(directions)
```

```

        moved = False
        for dx, dy in directions:
            # Enforce grid boundaries
            new_x = max(0, min(self.model.p.grid_size[0] - 1,
self.position[0] + dx))
            new_y = max(0, min(self.model.p.grid_size[1] - 1,
self.position[1] + dy))
            new_pos = (new_x, new_y)

            # Task 2: Avoidance Rule - check if target cell is already
occupied
            occupied_positions = [a.position for a in self.model.agents
if a != self]
            if new_pos not in occupied_positions:
                self.position = new_pos
                moved = True
                break

        # If surrounded/blocked by other agents, stay in place

# =====
# Model Definition
# =====
class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, AdvancedWalker)

        # Initialize non-overlapping starting positions
        grid_w, grid_h = self.p.grid_size
        all_positions = [(x, y) for x in range(grid_w) for y in
range(grid_h)]
        starting_positions = random.sample(all_positions,
min(self.p.agents, len(all_positions)))

        for agent, pos in zip(self.agents, starting_positions):
            agent.position = pos

        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        self.agents.step()

    def end(self):
        # Save final position to path history
        for agent in self.agents:
            agent.path.append(agent.position)

```



```

# =====
# Task 3: Spatial Analysis Function
# =====
def analyze_positions(model, label="Simulation"):
    final_positions = np.array([agent.position for agent in
model.agents])
    center = np.array([model.p.grid_size[0] / 2.0, model.p.grid_size[1]
/ 2.0])

    mean_pos = np.mean(final_positions, axis=0)
    std_pos = np.std(final_positions, axis=0)
    distances_from_center = np.linalg.norm(final_positions - center,
axis=1)
    avg_dist = np.mean(distances_from_center)

    print(f"--- Analysis Results [{label}] ---")
    print(f"Mean Position (X, Y): ({mean_pos[0]:.2f},
{mean_pos[1]:.2f})")
    print(f"Position Std Dev (X, Y): ({std_pos[0]:.2f},
{std_pos[1]:.2f})")
    print(f"Average Distance from Center: {avg_dist:.2f} units\n")

# =====
# Task 4: Compare Scenarios & Visualizations
# =====
def run_comparison():
    # Scenario A: Low density, short duration
    params_A = {'agents': 5, 'grid_size': (10, 10), 'steps': 20}
    model_A = RandomWalkModel(params_A)
    model_A.run()

    # Scenario B: Higher density, longer duration
    params_B = {'agents': 15, 'grid_size': (10, 10), 'steps': 50}
    model_B = RandomWalkModel(params_B)
    model_B.run()

    # Print distribution stats (Task 3)
    analyze_positions(model_A, "Scenario A (5 Agents, 20 Steps)")
    analyze_positions(model_B, "Scenario B (15 Agents, 50 Steps)")

    # Plot trajectories and final positions side-by-side
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))

    for ax, model, title in zip(axes, [model_A, model_B], ["Scenario A",
"Scenario B"]):
        ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
        ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)

```

```

ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True, linestyle='--', alpha=0.6)
ax.set_title(title)

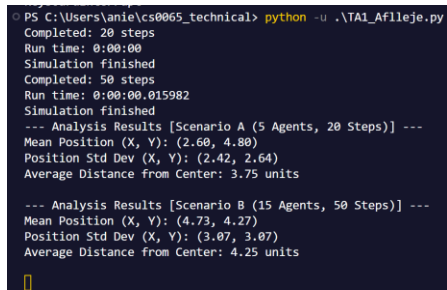
# Draw full trajectory paths and mark final points
for agent in model.agents:
    path_x, path_y = zip(*agent.path)
    ax.plot(path_x, path_y, alpha=0.6, linewidth=1.5)
    ax.scatter(path_x[-1], path_y[-1], s=80, zorder=5)

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    run_comparison()

```

Screenshots:

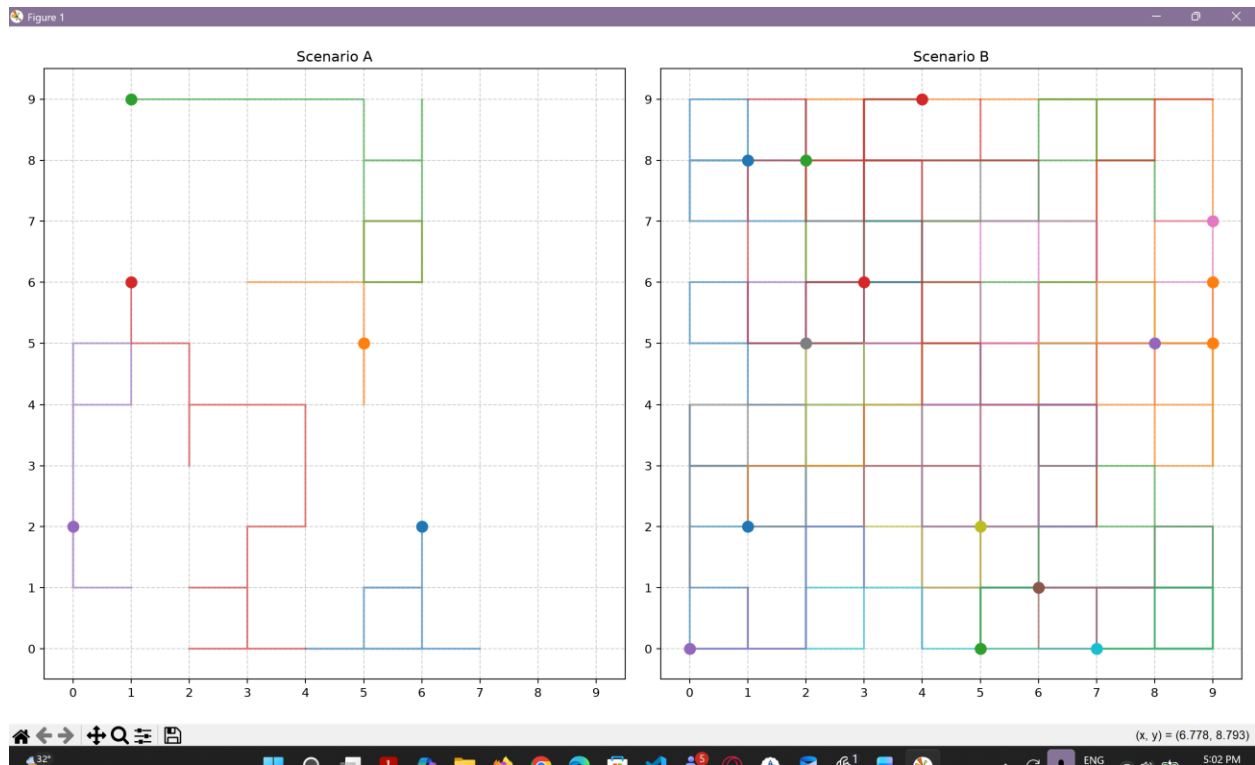


```

PS C:\Users\ania\cs0065_technical> python -u .\TA1_AfIleje.py
Completed: 20 steps
Run time: 0:00:00
Simulation finished
Completed: 50 steps
Run time: 0:00:00.015982
Simulation finished
--- Analysis Results [Scenario A (5 Agents, 20 Steps)] ---
Mean Position (X, Y): (2.60, 4.80)
Position Std Dev (X, Y): (2.42, 2.64)
Average Distance from Center: 3.75 units

--- Analysis Results [Scenario B (15 Agents, 50 Steps)] ---
Mean Position (X, Y): (4.73, 4.27)
Position Std Dev (X, Y): (3.07, 3.07)
Average Distance from Center: 4.25 units

```



Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
- 5.

Answer the Questions ↓

1. When the number of agents gets close to the number the grid can hold there is a lot of crowding. With rules that stop agents from running into each other agents often find their step blocked which makes it hard for them to move around and creates places where

movement slows down a lot. In models where there are no restrictions having many agents in the same area makes it hard to see what is happening because several agents are in the same spot, at the same time.

2. Increasing the time between steps lets the agents move from where they started. As time goes on the way they spread out changes. At first they are in groups close, to where they began. Later they start to spread out evenly across the whole area. This change makes the average distance they move squared go up. The -agent systems average distance squared increases.
3. No agents never move outside the grid. Boundary constraint logic guarantees that agents never move outside the grid by being implemented in the step() method:
`x = max(0, min(self.model.p.grid_size[0] - 1, x + direction[0]))`
`y = max(0, min(self.model.p.grid_size[1] - 1, y + direction[1]))`
therefore, the max(0, ...) prevents coordinates from dropping below 0, while min(grid_size - 1, ...) clamps values so they cannot exceed the upper boundary index. Any move attempting to cross the border is bounded back to the perimeter cell.
4. This simulation shows how things happen in life like how tiny particles in gas or liquid move around. It starts in areas where there are lots of molecules and spreads out randomly until it's evenly spread. That's called equilibrium. People often call this movement motion. It also helps show how people move in places like hallways or exits at a stadium. The model lets each person take random steps but they also try to avoid bumping into others nearby. This helps us see how bottlenecks form and how lines build up. In the study of diseases the simulation helps track how infections spread. If someone who is sick moves into the spot or next, to someone who's healthy the disease can pass. This is how researchers model contact networks to understand how illness moves through a group.